

CodeCraft: C++ 卡牌对战游戏 — 作业报告

小组成员：赵一鸣 游一诺 宋魏晨

2026 年 7 月 4 日

一、程序功能介绍

1.1 项目概述

CodeCraft 是一款以《杀戮尖塔》(Slay the Spire) 为灵感的卡牌对战游戏，核心创意在于将 C++ 面向对象编程的语法特性深度融合到卡牌机制中。每一张卡牌对应一个 C++ 编程概念，每一次出牌就是一次代码执行，玩家通过“编程”的方式进行战斗。

核心理念：

”函数即卡牌，对象即棋子，调用即出牌。”

1.2 核心玩法

游戏采用回合制卡牌对战的形式，玩家和敌人具有 `attack()`, `takeDamage()`, `summon()` 等基础函数，玩家每打出一张卡牌，就相当于写入一段代码，可以修改/调用这些函数。出牌结束后将会展示代码的运行过程。

游戏采用地图设计，玩家可以在生成的 DAG 节点上行走，不同节点上有不同的敌人，打败敌人后玩家可以替换自己的至多 3 张手牌。玩家需要通过巧妙地手牌配合击败敌人，取得胜利。

1.2.1 双层卡牌体系

游戏采用独特的双层卡牌系统：

1. 函数牌 — 修改玩家的成员函数实现

- 攻击函数牌：升级 `attack()` 函数
- 受击函数牌：升级 `takeDamage()` 函数
- 构造函数牌：升级 `summon()` 函数
- 复制构造牌：升级 `copySummon()`
- 移动构造牌：升级 `moveSummon()`
- 析构函数牌：升级 `sacrifice()`
- 逃跑函数牌：升级 `escape()` 函数

2. 指令牌 — 执行实际的战斗动作

- 攻击指令：如普通攻击、全力一击
- 防御指令：如防御姿态、坚守、治疗波
- 召唤指令：如召唤、快速复制
- 献祭指令：如血祭
- 特殊指令

3. 模板牌 (暂未实现)

- `ForceInline`：降低费用
- `Double/Triple`：效果触发多次

1.2.2 战斗流程

1. 回合开始 → 抽取 5 张手牌
2. 主要阶段 → 消耗能量打出手牌，打出函数牌升级能力，打出指令牌执行动作
3. 敌人行动 → 敌人根据 AI 策略进行攻击/防御/召唤
4. 回合结束 → 状态结算（灼烧、中毒、回复等）

1.3 主要功能模块

1. 战斗系统 — 回合制战斗，玩家 vs 敌人
2. 卡牌系统 — 三大类型卡牌、抽牌系统实现
3. 敌人系统 — 多种敌人 AI，包括 2 个 Boss
4. 仆从系统 — 玩家可召唤仆从协同作战
5. 状态系统 — 灼烧、中毒、冻结等状态
6. 地图系统 — DAG 地图，节点式关卡推进
7. 代码可视化 — 实时显示代码执行过程
8. 奖励系统 — 战斗胜利后可交换卡牌

二、项目各模块与类设计细节

2.1 整体架构

```
CodeCraft/
game/      # 游戏逻辑层（纯 C++，零 Qt 依赖）
  player.cpp
  enemy.cpp
  minion.cpp
  cards.cpp
  battle.cpp
  game_manager.cpp
CodeCraft/ # 头文件 + UI 层
*.h        # 头文件
mainwindow.cpp # Qt UI 主窗口
mainwindow.ui # UI 布局文件
main.cpp    # 程序入口
resources.qrc # 资源文件
```

2.2 核心类设计

2.2.1 Player 类 — 玩家

职责：管理玩家状态、仆从、可替换的 7 个核心函数

关键设计：

```
1  class Player {
2  public:
3      // 7 个可被函数牌替换的核心函数（使用 std::function 存储）
4      using AttackFunc = std::function<void(Enemy&, std::function<int(int)>>);
5      using TakeDamageFunc = std::function<void(int, DamageType)>;
6      using SummonFunc = std::function<Minion()>;
7      using CopySummonFunc = std::function<Minion(const Minion&)>;
8      using MoveSummonFunc = std::function<Minion(Minion&&)>;
9      using SacrificeFunc = std::function<void(Minion&)>;
10     using EscapeFunc = std::function<bool()>;
11
12     // 对外调用接口（永不改变，内部转发到可替换的 Impl）
13     void attack(Enemy& target, std::function<int(int)> damageModifier = nullptr);
14     void takeDamage(int dmg, DamageType type = DamageType::PHYSICAL);
15     Minion summon();
16     // ... 其他函数
17
18     // 函数牌通过 setter 替换函数实现
19     void setAttackFunc(AttackFunc f);
20     void setTakeDamageFunc(TakeDamageFunc f);
21     // ... 其他 setter
22
23 private:
24     // 7 个可替换的函数实现
25     AttackFunc attackImpl;
26     TakeDamageFunc takeDamageImpl;
27     // ...
28
29     // 玩家状态
30     int hp, maxHp;
31     int energy, maxEnergy;
32     int shield;
33     std::vector<Status> statuses;
34     std::vector<Minion> minions;
35 };
```

- 使用 std::function 实现函数可替换性
- 函数牌可以获取旧函数并包装新逻辑，实现函数叠加效果
- 提供 UI 回调机制（如 onHpChanged, onStatusAdded），实现逻辑层与 UI 层解耦

2.2.2 Enemy 类 — 敌人

职责：敌人基类，定义通用接口，子类实现不同 AI

关键设计：

```
1  class Enemy {
2  public:
3      virtual ~Enemy() = default;
4
5      // 纯虚函数：子类必须实现 AI 逻辑
6      virtual void takeTurn(Player& player) = 0;
7
8      // 意图系统：显示下回合行动
9      EnemyIntent nextIntent;
10
11     // 状态管理
12     std::vector<Status> statuses;
```

```

13         int hp, maxHp, shield, baseAttack;
14
15     protected:
16     void setIntent(EnemyIntent::Type t, int val = 0);
17 };
18
19 // 具体敌人实现
20 class Goblin : public Enemy {
21 public:
22     void takeTurn(Player& player) override; // 简单攻击 AI
23 };
24
25 class TemplateKing : public Enemy {
26 public:
27     void takeTurn(Player& player) override; // 复杂 Boss AI
28 private:
29     enum class Phase { FIRST, SECOND, THIRD };
30     enum class Mode { ATTACK, DEFENSE };
31     Phase currentPhase;
32     Mode currentMode;
33 };

```

- 策略模式：不同敌人通过继承实现不同 AI 策略
- 意图系统：提前显示敌人下回合行动，增加策略性
- Boss 设计具有多阶段、模式切换等复杂机制

2.2.3 Card 类 — 卡牌

职责：卡牌基类及三大子类型

关键设计：

```

1 // 基类
2 class Card {
3 public:
4     virtual ~Card() = default;
5     virtual void play(Player& player, Enemy* target = nullptr) = 0;
6     virtual Card* clone() const = 0;
7     virtual std::vector<std::string> getCodeLines() const;
8
9     std::string name, description;
10    int cost;
11    CardType type; // FUNCTION / COMMAND / TEMPLATE
12    Rarity rarity; // COMMON / UNCOMMON / RARE / LEGENDARY
13    TargetMode targetMode;
14 };
15
16 // 函数牌
17 class FunctionCard : public Card {
18 public:
19     FunctionTarget target; // ATTACK / TAKE_DAMAGE / SUMMON 等
20     virtual void play(Player& player, Enemy* enemy) = 0;
21 };
22
23 // 指令牌
24 class CommandCard : public Card {
25 public:
26     virtual void play(Player& player, Enemy* target) = 0;
27 };
28
29 // 模板牌
30 class TemplateCard : public Card {
31 public:
32     void setWrappedCard(FunctionCard* card); // 包装函数牌
33     virtual void applyWrapper(Player& player, Enemy* target) = 0;
34 protected:
35     FunctionCard* wrappedCard;
36 };

```

- Card → FunctionCard/CommandCard/TemplateCard → 具体卡牌
- 每张具体卡牌独立实现 play() 方法
- 提供 getCodeLines() 用于代码可视化
- 模板牌采用装饰器模式，包装函数牌增强效果

2.2.4 GameManager 类 — 游戏管理器

职责：回合流程、能量管理、牌组管理、地图系统

关键设计：

```

1 class GameManager {
2 public:
3     // 静态成员：跨实例共享
4     static GameMap currentMap;
5     static int currentNodeId;
6     static std::vector<std::unique_ptr<Card>> cardCollection;
7
8     // 地图生成（工厂方法）
9     static void generateMap(int seed);
10
11    // 卡牌工厂
12    static std::unique_ptr<Card> createCardByName(const std::string& name);
13    static std::unique_ptr<Enemy> createEnemyByName(const std::string& name);

```

```

14
15 // 实例成员：单局游戏状态
16 Player player;
17 BattleContext battle;
18 std::vector<std::unique_ptr<Card>> drawPile;
19 std::vector<std::unique_ptr<Card>> hand;
20 std::vector<std::unique_ptr<Card>> discardPile;
21
22 // 回合流程
23 void beginTurnWithoutDraw();
24 TurnResult finishTurnAfterCodeExecution();
25
26 // 牌组操作
27 DrawResult drawOneCard();
28 PlayResult playCardAsCode(int handIndex, Enemy* target);
29 };

```

- 根据字符串名称创建卡牌和敌人实例
- 静态成员实现地图和卡牌库跨战斗保持
- 将出牌转换为代码命令队列，逐个执行并可视化

2.2.5 BattleContext 类 — 战斗上下文

职责：管理战斗中的敌人列表，提供战斗查询接口

关键设计：

```

1 class BattleContext {
2 public:
3     std::vector<std::unique_ptr<Enemy>> enemies;
4
5     bool allEnemiesDead() const;
6     bool isBattleOver() const;
7     Enemy* firstAliveEnemy() const;
8     std::vector<Enemy*> aliveEnemies() const;
9 };

```

- 记录战斗信息，提供各种接口

2.2.6 Minion 类 — 仆从

职责：玩家召唤的战斗单位

关键设计：

```

1 class Minion {
2 public:
3     std::string name;
4     int hp, maxHp, attack, shield;
5     MinionType type; // NORMAL / ELITE
6     std::vector<Status> statuses;
7
8     void takeDamage(int dmg, DamageType dtype);
9     void addStatus(Status s);
10    int getEffectiveAttack() const; // 考虑状态加成
11 };

```

2.2.7 MainWindow 类 — UI 主窗口

职责：Qt UI 层，负责显示和用户交互

关键功能：

- 卡牌手牌显示和点击交互
- 代码编辑器显示（玩家代码、敌人代码）
- 实时代码高亮和执行动画
- 血量、护盾、能量条显示
- 仆从和敌人的信息显示
- 地图界面、帮助界面
- 通过回调函数响应游戏逻辑层事件
- 动画系统：卡牌飞行、代码高亮、伤害飘字等
- 代码可视化：实时显示代码执行过程

2.3 设计模式应用总结

设计模式	应用位置	作用
策略模式	Enemy 子类	不同敌人实现不同 AI 策略
工厂模式	GameManager	根据名称创建卡牌和敌人
装饰器模式	TemplateCard	包装函数牌增强效果
观察者模式	UI 回调机制	游戏逻辑通知 UI 更新
单例模式	GameManager 静态成员	地图和卡牌库全局唯一
状态模式	Status 系统	不同状态类型有不同行为

2.4 关键数据结构

2.4.1 枚举类型系统

```

1 // types.h 定义了游戏的类型系统
2 enum class CardType { FUNCTION, COMMAND, TEMPLATE }; // 卡牌类型
3 enum class Rarity { COMMON, UNCOMMON, RARE, LEGENDARY }; // 卡牌稀有度
4 enum class TargetMode { NONE, SINGLE_ENEMY, ALL_ENEMIES, SINGLE_ALLY, SELF }; // 卡牌目标类型
5 enum class DamageType { PHYSICAL, FIRE, ICE, LIGHTNING, SHADOW, HOLY, POISON }; // 伤害类型 (具体作用待之后
   加入改进)
6 enum class StatusType { BURN, POISON, FREEZE, STUN, WEAKEN, STRENGTH, ... }; // 状态类型
7 enum class FunctionTarget { ATTACK, TAKE_DAMAGE, SUMMON, COPY_SUMMON, ... }; // 函数类型

```

2.4.2 DAG 地图结构

```

1 struct MapNode {
2     int id, depth;
3     bool isStart, isBoss;
4     std::vector<std::string> enemyTypes;
5     std::vector<std::unique_ptr<Card>> rewardCards;
6 };
7
8 struct GameMap {
9     int seed;
10    std::vector<MapNode> nodes;
11    std::vector<std::vector<int>> edges; // 邻接表
12    int startNodeId, bossNodeId;
13 };

```

三、小组成员分工情况

成员	负责模块
赵一鸣	UI 界面设计与实现 (MainWindow)、动画设计、角色立绘设计、代码可视化、游戏的测试与调整
游一诺	战斗逻辑、地图系统的设计与实现 (GameManager)、战斗上下文记录、类型设计
宋魏晨	卡牌的设计与实现、Enemy 设计与实现、游戏数值调整

四、项目总结与反思

4.1 项目成果

较为完整的实现了游戏的卡牌战斗系统、敌人设计、UI 界面设计等系统，完成了一个具有一定可玩性、能够正常运行的游戏制作。项目创意的提出了“函数即卡牌”的概念，将打出卡牌转化为写入代码，将游戏娱乐与程序设计知识有机结合。

4.2 不足之处

1. 测试不足：主要依赖手动测试
2. 卡牌实现：模板牌和一些卡牌因为游戏系统设计问题暂未实现
3. 平衡性：部分卡牌强度过高，仆从类强度过低，游戏具有随机性，需要更多测试调整
4. UI 美术：美术资源较为简陋，可以提升视觉效果
5. 存档系统：游戏进度无法保存，需要一局打完
6. 奖励系统：游戏暂未引入奖励与遗物系统

4.3 学习收获

1. 面向对象设计：深刻理解了继承、多态、虚函数的实际应用场景
2. 现代 C++：熟悉了 c++ 较为线代的智能指针、Lambda、std::function 等特性
3. 软件工程：体会到模块化设计、接口解耦的重要性
4. 团队协作：学会了 Git 协作、代码 Review、任务分工，提升了团队协作能力

4.4 结语

本项目成功地将 C++ 编程概念与卡牌游戏深度融合，既实现了完整的游戏玩法，又充分展示了 C++ 面向对象编程的核心特性。通过这个项目，我们不仅提升了编程能力，更重要的是学会了如何将抽象的编程概念转化为具体的游戏机制，体会到了“寓教于乐”的设计思想。

这是一次富有挑战性和创造性的开发经历，为我们今后的软件开发学习打下了坚实的基础。

项目地址：<https://github.com/MEKHANE114514/Slay-The-Spire-Clone>